Subject: Deep Learning
SID: 2110826142

Assignment: 02

Code:

```
import torch
import torch.nn as nn
import torchvision.datasets as datasets
import torchvision.models as models
from torch.utils.data import DataLoader
import numpy as np
import matplotlib.pyplot as plt
import os

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap

MODEL_NAME = "resnet50" # for others use different model.

BASE_PATH = "....."
MY_FACE_PATH = "....."
FACE_DATA_PATH = "....."
RESULT_PATH = "....."
BATCH_SIZE = 32
EPOCHS = 5
```

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

def apply_img_transform():
    return transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225])
    ])

def load_actual_data(transform):
    train = datasets.ImageFolder(FACE_DATA_PATH, "train"), transform=transform)
    val = datasets.ImageFolder(FACE_DATA_PATH, "val"), transform=transform)
    train_loader = DataLoader(train, 32, True)
    val_loader = DataLoader(val, 32, True)
```

```
    return train_loader, val_loader, train_classes

def load_my_faces(transform):
    dataset = datasets.ImageFolder(MY_FACE_PATH,
                                   transform)
    loader = DataLoader(dataset, batchsize=32,
                        shuffle=False)
    return loader, dataset.targets

from torchvision.models import resnet50, vgg16, mobile
                                                -net V2
from torchvision.models import RESNET50_Weights . . .

def get_desired_model(model_name, num_classes=None,
                      pretrained=True):
    if model_name, num_of_class = None == "resnet50"
        weights = ResNet50_Weights.Default
        model = resnet50(weights)
        in_features = model.fc.in_features
```

```
if num_classes:
    model.fc = nn.Linear(in_features, num_classes)

feature_extractor = nn.Sequential()

return model.to(device), feature_extractor.to(device)

def extract_img_features(model, train_loader, ...
    criteria = nn.CrossEntropy()
    optimizer = torch.optim.Adam(model.param...
                                 lr = 1e-4)
    model.train()
    for epochs in range(epochs):
        total_loss = 0
        for imgs, labels in train_loader:
            imgs, labels = img.to, labels.to
            optimizer.zero_grad()
            outputs = model(imgs)
```

```
        loss += criteria(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

def dimension_reduction(features):
    results = {}
    results['pca'] = PCA(n_comp=2).fit_transform(features)
    n_samples = features.shape[0]
    perplexity = max(2, min(30, n_samples/13))
    results['tsne'] = TSNE(n_comp=2, perplexity
                           ).fit_transform(features)
    results['umap'] = umap.UMAP(n_comp=2).
                      fit_transform(features)
    return results.
```

```
def save_plots(embedding, labels, title, save_path):
    plt.figure()
    plt.scatter(embedding[:, 0], embedding[:,
                c=labels)
    plt.title(title)
    plt.savefig(save_path)

def main():
    transform = apply_img_transform()
    train_ld, val_ld, classes = load_actual_face
                                (transform)
    model_result_path = (RESULT_PATH, RESULT
                         _NAME)
    base_model, feature_extractor = get_model
                                    (MODEL_NAME, True
    feature_before, labels = extract_img_feature
                             feature_extractor, my_loader)
```

```
reduced_before_train = reduce_dimensions(features_before)

for method, emb in reduced_before.items():
    save_plot(
        emb, labels
        f"{Model_Name}_before_{method}"
    )

model_ft, _ = get_model(MODEL_Name, num_ftrs, len(classes))

feature_extractor_ft = nn.Sequential(*list(model_ft.
                        children())[:-1])

feature_extractor.eval()

feature_after, labels = extract_features(feature_
                        extractor_ft, my_loader)

for method, emb in reduced_items():
    save_plot( emb, labels, f"{Model Name} Afte
                {method})

main()
```

Report:

Data collection: Face data was collected from kaggle fine face dataset containing a total of 750 images spread to 5 classes.

Class names: trump, modi, gates, jack, musk

Custom face images collected by me was collected by the consent from my friends. This dataset contained 5 classes each representing facial images of 5 individuals.

Flower training data was collected from CIFAR-100 dataset that contains a total of 100 classes. Custom captured flower images belonging to 5 distinct flower class was used for testing.

Methodology: The collected data sets were divided into two groups. The data collected from internet was used for training pretrained models like resnet 50, mobilenet V2 etc. And the custom images were used for testing only. Weights were assigned based on the pretrained model. And all the specified tasks were conducted accordingly.

Result: Based on the result and before vs after comparison I have found that none of the models were good enough to extract distinctive features as the training epochs were low due to hard ware constraints. The attached drive contains resulting images from both before vs after comparison